Questions techniques

Les questions probables du jury. Les réponses ne répètent pas l'oral : elles donnent le détail qui se cache derrière. Le numéro de diapositive indique ce qui a déjà été dit.

Données et sélection

1 Pourquoi ne pas avoir téléchargé toutes les peintures ?

Suite de la diapositive 9

Le serveur est lent : environ **0,7 image par seconde**. Les 32 000 peintures auraient demandé plus de 12 heures, et beaucoup auraient été retirées ensuite (vues de détail, fresques).

À retenir : 0,7 image/s : on ne télécharge que l'utile.

---

2 Que faites-vous quand un genre a moins d'images que sa part ?

Suite de la diapositive 12

On le prend **en entier**, et ce qui reste est partagé entre les autres genres. Par exemple, les études (moins de 80 candidates) sont toutes prises. Les plus grands genres reçoivent le reste de la division. Le total fait toujours 8 000.

À retenir : Petits genres en entier, le reste partagé.

---

3 Comment être sûr qu'une œuvre n'est pas à la fois en entraînement et en test ?

Suite de la diapositive 12

Un tableau et ses vues de détail ont la même clé d'œuvre (**work_key**). Le côté est choisi en **hachant cette clé** (SHA-1), pas au hasard : il ne dépend ni de l'ordre des lignes ni de la graine. Sans cela, le FID serait faussement bon.

À retenir : Découpage par œuvre, pas par image.

---

4 Pourquoi 1 141 téléchargements ont-ils échoué ?

Suite de la diapositive 12

Pages ou fichiers introuvables, et un serveur irrégulier. Chaque image est essayée **3 fois** au maximum. Le manifeste garde le statut de chaque image : le téléchargement peut s'arrêter et reprendre sans tout refaire. 80 doublons exacts ont aussi été ignorés.

À retenir : 3 essais, et un manifeste pour reprendre.

---

5 Comment fonctionnent concrètement vos deux contrôles de doublons ?

Suite de la diapositive 14

L'**empreinte** réduit l'image à 8×8 en gris et note chaque pixel plus clair ou plus sombre que la moyenne : 64 bits. Deux images sont candidates si au plus 2 bits diffèrent. La **confirmation** compare les vrais pixels : écart moyen inférieur à 10 sur 255.

À retenir : 64 bits pour trouver, les pixels pour confirmer.

---

6 Pourquoi recadrer plutôt qu'étirer l'image ou ajouter des bandes ?

Suite de la diapositive 13

Étirer **déforme** les visages. Ajouter des bandes noires apprendrait au GAN à peindre des bandes. Le recadrage perd un peu des bords, mais le sujet est le plus souvent au centre.

À retenir : Ni déformation, ni bandes.

---

7 Pourquoi seulement le miroir horizontal ?

Suite de la diapositive 13

Le GAN **reproduit aussi les augmentations**. Avec des rotations ou des changements de couleur, il peindrait des tableaux penchés ou aux couleurs fausses. Un tableau inversé gauche-droite reste plausible.

À retenir : Ce qu'on augmente, le GAN l'imite.

---

8 Dans quel ordre est votre pipeline, et pourquoi l'ordre compte-t-il ?

Suite de la diapositive 20

Resize, CenterCrop, RandomHorizontalFlip, ToTensor, puis **Normalize(0,5 ; 0,5)**. Normalize travaille sur un tenseur entre 0 et 1 : placé avant ToTensor, il échoue. Le résultat va de −1 à 1, comme la sortie du générateur.

À retenir : Normalize toujours après ToTensor.

---

9 Quels biais restent dans vos données ?

Suite de la diapositive 10

Au-delà du religieux : peinture **européenne** (italienne à 42 %), du 15e au 19e siècle, surtout des artistes hommes. Le GAN ne peut pas peindre ce qu'il n'a jamais vu, comme l'art abstrait ou non européen.

À retenir : Le modèle hérite des biais du catalogue.

Architecture du modèle

10 Combien de canaux et de paramètres ont vos réseaux ?

Suite de la diapositive 15

Le générateur commence à **512 canaux** en 4×4 et divise par deux à chaque bloc : 256, 128, 64, puis 3 (RVB). Environ **3,6 millions** de paramètres. Le discriminateur fait l'inverse, de 64 à 512 canaux : **2,8 millions**. Ni pooling ni couche dense : uniquement des convolutions.

À retenir : 512 → 3 canaux ; 3,6 M et 2,8 M paramètres.

11 Pourquoi pas de sigmoïde à la fin du discriminateur ?

Suite de la diapositive 15

La perte **BCEWithLogitsLoss** applique elle-même la sigmoïde, en un seul calcul plus **stable** (pas de log(0)). La sigmoïde n'est ajoutée que pour afficher une probabilité, dans l'application.

À retenir : Sigmoïde intégrée à la perte.

12 D'où venait le damier de votre première version ?

Suite de la diapositive 19

Des **convolutions transposées** : quand leurs filtres se chevauchent de façon inégale, certains pixels sont plus « peints » que d'autres. L'effet s'atténue avec l'entraînement. L'alternative : un agrandissement simple suivi d'une convolution normale.

À retenir : Damier = chevauchement des convolutions transposées.

13 Où avez-vous mis BatchNorm, et où non ?

Non abordé à l'oral

Dans tous les blocs, **sauf** la sortie du générateur et l'entrée du discriminateur : ce sont les vraies valeurs des pixels, il ne faut pas les renormaliser. L'article DCGAN montre que sinon l'entraînement devient instable.

À retenir : Pas de BatchNorm sur les pixels.

14 Pourquoi LeakyReLU dans le discriminateur ?

Non abordé à l'oral

ReLU met à zéro les valeurs négatives et leur gradient. **LeakyReLU** (pente 0,2) laisse passer un petit gradient. C'est important : le générateur apprend **à travers** le discriminateur.

À retenir : Le gradient du générateur passe par D.

15 Comment les poids sont-ils initialisés ?

Non abordé à l'oral

Loi normale de moyenne 0 et d'écart-type **0,02** pour les convolutions ; autour de 1 pour BatchNorm. C'est la recette de l'article DCGAN.

À retenir : N(0 ; 0,02).

---

16 Qu'avez-vous changé pour passer en 128×128 ?

Suite de la diapositive 21

Un bloc de plus dans chaque réseau. Le générateur passe à 12,8 millions de paramètres, **3,5 fois plus**. Le code reste le même : seule la taille d'image change.

À retenir : Un bloc de plus, 3,5× plus de paramètres.

Entraînement

17 Pourquoi garder les hyperparamètres de l'article ?

Suite de la diapositive 16

Ils sont connus pour être stables. En les gardant, nous ne changions **qu'une chose à la fois** : les données. Ainsi, nous savons d'où vient chaque amélioration. Pour β1 : il règle la mémoire des gradients passés. Dans un GAN, l'adversaire change à chaque pas, donc 0,5 au lieu de 0,9 rend l'optimiseur plus réactif.

À retenir : Une seule variable à la fois : les données.

---

18 Pourquoi le générateur vise-t-il « vrai » au lieu de minimiser l'erreur du discriminateur ?

Suite de la diapositive 15

La formule d'origine, minimiser log(1 − D(G(z))), donne des **gradients presque nuls** au début, quand le discriminateur rejette tout facilement. Viser « vrai = 1 » (perte **non saturante**) donne au contraire un fort gradient quand le générateur est mauvais.

À retenir : Non saturante : il apprend même au début.

---

19 Techniquement, comment figez-vous le générateur pendant l'étape du discriminateur ?

Suite de la diapositive 5

Avec **fake.detach()** : le gradient ne remonte pas dans le générateur. À l'étape suivante, les mêmes images passent sans detach, et seul l'optimiseur du générateur fait un pas.

À retenir : detach() coupe le gradient.

---

20 Quand une courbe de perte doit-elle vous inquiéter ?

Suite de la diapositive 17

Si la perte du discriminateur tombe à **0** : il a gagné, le générateur ne reçoit plus d'information. Si une valeur devient **infinie** : divergence, notre code s'arrête alors. Mais une perte ne dit pas si les images sont belles : seuls le FID et l'œil le disent.

À retenir : La perte surveille, elle ne juge pas.

---

21 Combien de temps a pris l'entraînement ?

Suite de la diapositive 16

Sur processeur, une époque de 6 100 images prend environ **90 secondes**. Les portraits ont tourné 180 époques sur 3 347 images : **4 680 itérations**, presque autant que la baseline (4 700). C'est ce qui rend les deux comparables. Le code utilise automatiquement un GPU s'il en trouve un (Colab).

À retenir : Même nombre d'itérations pour comparer.

---

22 Et si l'entraînement plante ?

Non abordé à l'oral

À chaque époque, un **point de reprise** complet : poids, optimiseurs, historique et états du hasard. Il est écrit à côté, relu pour vérifier qu'il n'est pas corrompu, puis remplace l'ancien. Le script relance l'entraînement 5 fois au maximum, et il reprend où il s'était arrêté.

À retenir : Sauvegarde vérifiée à chaque époque.

---

23 Vos résultats sont-ils reproductibles ?

Non abordé à l'oral

Oui : graine fixe (42) pour Python, NumPy et PyTorch, sauvegardée avec le point de reprise ; sélection et découpage déterministes. Réserve : sur GPU, certaines opérations ne le sont pas parfaitement.

À retenir : Graine fixe + découpage déterministe.

Évaluation

24 Comment calculez-vous le FID ?

Suite de la diapositive 18

Chaque image passe dans **Inception v3** (entraîné sur ImageNet), qui la résume en **2 048 nombres**. Les vraies et les fausses images forment deux nuages, vus comme des gaussiennes. Le FID additionne l'écart des **moyennes** et l'écart des **covariances**. Nos images sont agrandies en 299×299, la taille attendue par Inception.

À retenir : Distance entre deux nuages de caractéristiques.

---

25 Pourquoi votre meilleur FID possible est-il 62 et pas 0 ?

Suite de la diapositive 18

Le FID est **biaisé par la taille de l'échantillon** : avec 657 images, deux ensembles de vraies peintures ne sont jamais à 0. C'est aussi pour cela que nous ne comparons pas à des FID publiés, calculés sur 50 000 images.

À retenir : Petit échantillon = FID gonflé.

---

26 Pouvez-vous comparer directement 211 (tous genres) et 204 (portraits) ?

Suite de la diapositive 21

Non : les deux FID ne sont **pas calculés sur le même jeu de test** (657 images contre 390). Chaque jeu a sa propre référence (62 et 73). C'est pour cela que nous comparons les **rapports**, ×3,4 et ×2,8, et non les FID bruts.

À retenir : On compare les rapports, pas les FID bruts.

---

27 Quelles sont les autres limites du FID ?

Non abordé à l'oral

Inception a appris sur des **photos**, pas sur des peintures. L'agrandissement de 64 à 299 pixels change les images. Et un seul nombre ne dit pas si le problème vient de la qualité ou de la diversité.

À retenir : Appris sur des photos, un seul nombre.

---

28 Qu'apporte le KID par rapport au FID ?

Suite de la diapositive 19

Il compare les mêmes caractéristiques, sans supposer de gaussienne (noyau polynomial). Son estimateur est **non biaisé** : fiable même avec peu d'images. Moyenné sur 50 tirages : baseline 0,150 ; portraits 0,135 ; vraies images ≈ 0.

À retenir : Non biaisé : adapté aux petits jeux.

---

29 Votre KID de référence est négatif pour les portraits. Erreur ?

Non abordé à l'oral

Non. Un estimateur non biaisé peut passer **légèrement sous zéro** quand la vraie valeur est 0 : c'est du bruit. −0,0001 veut dire « les deux ensembles de vraies images sont identiques en distribution ».

À retenir : Non biaisé : il peut passer sous zéro.

---

30 Comment mesurez-vous « la peinture la plus proche » ?

Suite de la diapositive 18

Par la **similarité cosinus** des caractéristiques Inception, sur toutes les images d'entraînement. En moyenne 0,75 (une copie donnerait 1). Limite à reconnaître : nous l'avons fait pour **6 images** seulement.

À retenir : Cosinus 0,75 : proche, pas copié.

Application et code

31 Que charge exactement l'application ?

Suite de la diapositive 22

Un fichier de **poids seulement** (environ 14 Mo), sans les optimiseurs ni l'historique. Il est chargé une seule fois grâce à **st.cache_resource**. Le numéro de l'œuvre sert de graine pour tirer les 100 nombres.

À retenir : Poids seuls, chargés une fois.

---

32 La note du discriminateur est-elle une mesure de qualité ?

Suite de la diapositive 22

Non, c'est une **illustration**. Le discriminateur a été entraîné contre ce générateur précis, et à l'équilibre il est censé hésiter. Pour la qualité, nous utilisons le FID et le KID.

À retenir : Pédagogique, pas une métrique.

---

33 Comment avez-vous testé votre code ?

Non abordé à l'oral

Avec **pytest**, une dizaine de fichiers. Par exemple : la taille des images en sortie du générateur, la reprise d'un entraînement, le découpage déterministe, un FID nul pour deux ensembles identiques, la même peinture pour une même graine.

À retenir : Tests automatiques avec pytest.

Pour aller plus loin

34 Concrètement, que changerait le WGAN-GP ?

Suite de la diapositive 24

Le discriminateur devient un **critique** qui donne un score, sans sigmoïde. La perte estime la **distance de Wasserstein** entre vraies et fausses images, et une **pénalité de gradient** garde le critique régulier. L'entraînement est plus stable, et la perte suit mieux la qualité.

À retenir : Plus stable, perte plus parlante.

---

35 Comment feriez-vous un GAN conditionnel ?

Suite de la diapositive 24

On donne au générateur, en plus du bruit, une **étiquette** : le genre (portrait, paysage…). Le discriminateur la reçoit aussi. Nos données ont déjà cette colonne (TYPE) : on pourrait choisir le genre dans l'application.

À retenir : Bruit + genre = on choisit ce qu'on peint.

---

36 Pourquoi pas StyleGAN ou un modèle de diffusion ?

Non abordé à l'oral

Ils donnent de bien meilleures images, mais demandent des **GPU puissants** pendant des jours, et souvent plus de données. Nous voulions une chaîne complète et comprise de bout en bout.

À retenir : Meilleurs, mais trop coûteux ici.

---

37 Avez-vous le droit d'utiliser ces images ?

Suite de la diapositive 8

La Web Gallery of Art autorise un usage **éducatif et personnel**. Les œuvres anciennes sont dans le domaine public, mais pas forcément leurs photographies. Notre dépôt reste donc privé.

À retenir : Usage éducatif, dépôt privé.
